

# Out-of-the-Box DQ rules for Product 360

Informatica MDM - Product 360

Version: 10.5 HotFix 5

# Table of Contents

1 Project Structure within Informatica Data Quality (Design Environment) .....7
1.1 Upgrading from one IDQ Version to another .....8
2 Importing Rules into Product 360 .....10
3 Rule types and reference tables .....10
3.1 Generic rules .....10
3.2 Structure Feature / Attribute rules .....11
3.3 Product code validation rules .....11
3.4 Classification rules .....11
3.5 Lookup and standardization rules .....11
3.6 Reference tables .....11
4 Generic Rules .....12
4.1 Check_DependentFields .....12
4.1.1 General description .....12
4.1.2 Input ports .....12
4.1.3 Output ports .....12
4.1.4 Meta data and reference tables .....12
4.1.5 Example usage .....13
4.1.5.1 Example output .....13
4.2 Check_Equal .....13
4.2.1 General description .....13
4.2.2 Input ports .....13
4.2.3 Output ports .....13
4.2.4 Meta data and reference tables .....14
4.2.5 Example usage .....14
4.2.5.1 Example output .....14
4.3 Check_isEmpty .....14
4.3.1 General description .....14
4.3.2 Input ports .....14
4.3.3 Output ports .....14
4.3.4 Meta data and reference tables .....15

4.3.5 Example usage ..... 15
4.3.5.1 Example output ..... 15
4.4 Check_isEmpty_SubentityLevel ..... 15
4.4.1 General description ..... 15
4.4.2 Input ports ..... 15
4.4.3 Output ports ..... 16
4.4.4 Meta data and reference tables ..... 16
4.4.5 Example usage ..... 16
4.4.5.1 Example output ..... 16
4.5 Check_MaxLength ..... 16
4.5.1 General description ..... 16
4.5.2 Input ports ..... 17
4.5.3 Output ports ..... 17
4.5.4 Meta data and reference tables ..... 17
4.5.5 Example usage ..... 17
4.5.5.1 Example output ..... 17
4.6 Check_MinLength ..... 18
4.6.1 General description ..... 18
4.6.2 Input ports ..... 18
4.6.3 Output ports ..... 18
4.6.4 Meta data and reference tables ..... 18
4.6.5 Example usage ..... 18
4.6.5.1 Example output ..... 19
5 Feature / Attribute Rules ..... 20
5.1 Check_DataTypes ..... 20
5.1.1 General description ..... 20
5.1.2 Sub-Entity type for Input ..... 20
5.1.3 Input ports ..... 20
5.1.4 Output ports ..... 21
5.1.5 Meta data and reference tables ..... 21
5.1.6 Example usage ..... 22
5.1.6.1 Example output ..... 22
5.2 Check_MandatoryValues ..... 22
5.2.1 General description ..... 22

5.2.2 Sub-Entity type for Input ..... 23
5.2.3 Input ports ..... 23
5.2.4 Output ports ..... 24
5.2.5 Meta data and reference tables ..... 24
5.2.6 Example usage ..... 25
5.2.6.1 Example output ..... 25
5.3 Check_PresetValues ..... 25
5.3.1 General description ..... 25
5.3.2 Sub-Entity type for Input ..... 25
5.3.3 Input ports ..... 25
5.3.4 Output ports ..... 26
5.3.5 Meta data and reference tables ..... 26
5.3.6 Example usage ..... 27
5.3.6.1 Example output ..... 27
5.4 Check_MissingAttributes ..... 28
5.4.1 General description ..... 28
5.4.2 Sub-Entity type for Input ..... 28
5.4.3 Input ports ..... 28
5.4.4 Output ports ..... 28
5.4.5 Meta data and reference tables ..... 29
5.4.6 Example usage ..... 29
5.4.6.1 Example output ..... 29
6 Product Code Validation Rules ..... 30
6.1 Validate_GTIN ..... 30
6.1.1 General description ..... 30
6.1.2 Input ports ..... 30
6.1.3 Output ports ..... 30
6.1.4 Meta data and reference tables ..... 30
6.1.5 Example usage ..... 30
6.1.5.1 Example output ..... 31
6.2 Validate_UPC ..... 31
6.2.1 General description ..... 31
6.2.2 Input ports ..... 31
6.2.3 Output ports ..... 31

6.2.4 Meta data and reference tables ....31
6.2.5 Example usage ....31
6.2.5.1 Example output ....32
7 Classification Rules ....33
7.1 Classify_Product ....33
7.1.1 General description ....33
7.1.2 Input ports ....33
7.1.3 Output ports ....33
7.1.4 Meta data and reference tables ....34
7.1.5 Example usage ....34
7.1.5.1 Example output ....34
7.2 Identify_Language ....35
7.2.1 General description ....35
7.2.2 Input ports ....35
7.2.3 Output ports ....35
7.2.4 Meta data and reference tables ....35
7.2.5 Example usage ....35
7.2.5.1 Example output ....36
8 Lookup and Standardization Rules ....37
8.1 Check_Profanity ....37
8.1.1 General description ....37
8.1.2 Input ports ....37
8.1.3 Output ports ....37
8.1.4 Meta data and reference tables ....38
8.1.5 Example usage ....38
8.1.5.1 Example output ....38
8.2 Parse_Color ....38
8.2.1 General description ....38
8.2.2 Input ports ....39
8.2.3 Output ports ....39
8.2.4 Meta data and reference tables ....39
8.2.5 Example usage ....39
8.2.5.1 Example output ....40
8.3 Standardize_Color ....40

8.3.1 General description ....40
8.3.2 Input ports ....40
8.3.3 Output ports ....40
8.3.4 Meta data and reference tables ....41
8.3.5 Example usage ....41
8.3.5.1 Example output ....41
8.4 Standardize_CompanyName ....41
8.4.1 General description ....41
8.4.2 Input ports ....42
8.4.3 Output ports ....42
8.4.4 Meta data and reference tables ....42
8.4.5 Example usage ....42
8.4.5.1 Example output ....43
8.5 Standardize_UOM ....43
8.5.1 General description ....43
8.5.2 Input ports ....43
8.5.3 Output ports ....43
8.5.4 Meta data and reference tables ....44
8.5.5 Example usage ....44
8.5.5.1 Example output ....44
9 Reference Tables ....46
9.1 Product 360 administrative reference tables ....46
9.2 Reference tables for data look ups and standardizations ....49

**Out-of-the-Box DQ Rules for Product 360**

The Out-of-the-Box data quality rule package enables customers to use pre-built templates in order to get started quickly.

This document describes the use cases and set up for all standard data quality rules that are part of the Informatica Data Quality for Product 360 delivery.

# 1 Project Structure within Informatica Data Quality (Design Environment)

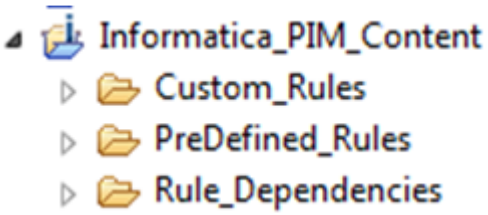
The out-of-the-box DQ rules for Product 360 package is structured into two files that make up the Informatica Data Quality project that can be used in Product 360:

1. Informatica_PIM_Content.xml
   This file contains the rules itself in the defined structure. See details on the rules and their functionality under "Rule Types"

2. Informatica_PIM_Content.zip
   This file contains all the meta data like demo classifiers for the auto classification of items and reference tables used by some of the rules. See details on the reference tables and their functionality under "Reference Tables"

Importing the Product 360 rules into Informatica Data Quality is only necessary if you want to add additional custom rules for Product 360.

When importing the XML and the ZIP file into Informatica Data Quality it should reflect the following structure:




Only Mapplets included in following 2 paths will be surfaced through the Product 360 UI:

- Informatica_PIM_Content**Custom_Rules**: Used for custom rules built by customer
- Informatica_PIM_Content**PreDefined_Rules**: Contains all pre-built / shipped rules for Product 360

Please make sure to put custom rules always under **Custom_Rules** as they won't be available through the UI if structured differently!

Without filtering of views, a Product 360 user is overwhelmed with the amount of mapplets displayed and has no indication which ones are ready for consumption in Product 360 and which ones are nested mapplets.

Place mapplets / objects that should not be surfaced to the user selection screen in Product 360 in **Rule_Dependencies** folder or other project.

On import of the ZIP file all content for the sub-folder Rule_Dependencies is transferred into the Informatica Data Quality project.

- Used for Product 360 specific rule dependencies – mapplets not shown to users, reference tables, content sets, etc.
- Use of other projects (e.g. Informatica_DQ_Content, custom project, etc.)
- Allowed and recommended to support single source of truth for rule design
- Limitation in Product 360 only applies to viewing rules from XML in the Product 360 User interface, all other dependencies are required during execution and included in exports

Make sure custom rules are exported with a **different XML filename than the standard rules (Informatica_PIM_Content.xml)**. You can have either one XML for all custom rules (e.g. “Custom_Rules_of_MyEnterprise.xml”) or even better one XML for each of them (“my_color_standardization.xml”, etc.). The export of custom rules should not include the **PreDefined_Rules** folder of the project so you can update the pre-built rules with any Product 360 upgrade and without any additional effort just by using the XML shipped with the install packages.

## 1.1 Upgrading from one IDQ Version to another

In case the embedded IDQ SDK deployed inside Product 360 has received a version upgrade it is also needed to upgrade the IDQ design environment containing your custom DQ rules (Informatica Developer). This can be achieved by performing an upgrade of the IDQ instance itself to the higher version while having the rules used for Product 360 in the IDQ repository. The upgrade of Informatica Developer will automatically upgrade the rules to that higher version as well.

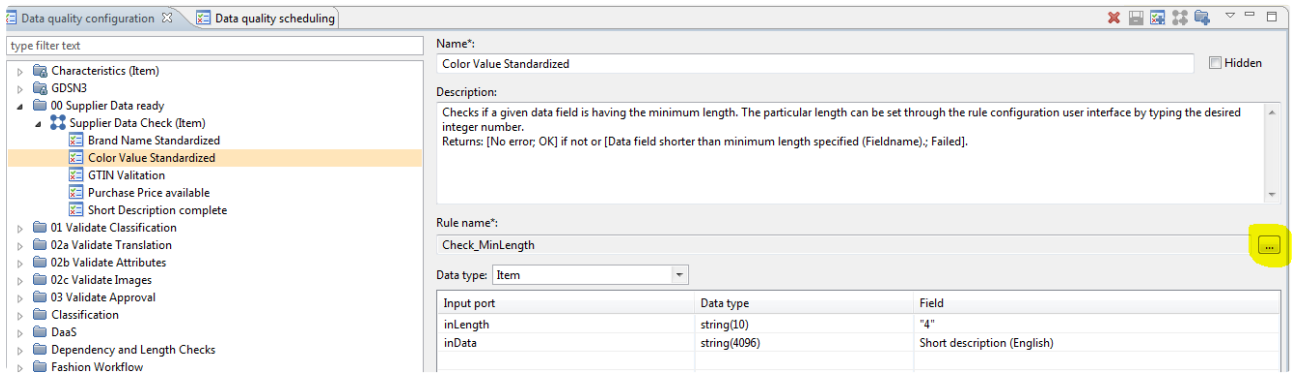
For more information on upgrading the Informatica Developer (IDQ Server) to a higher version please refer to the standard documentation of Informatica Data Quality.

Next step is now to export your custom rules and potentially additional reference data as XML and ZIP.

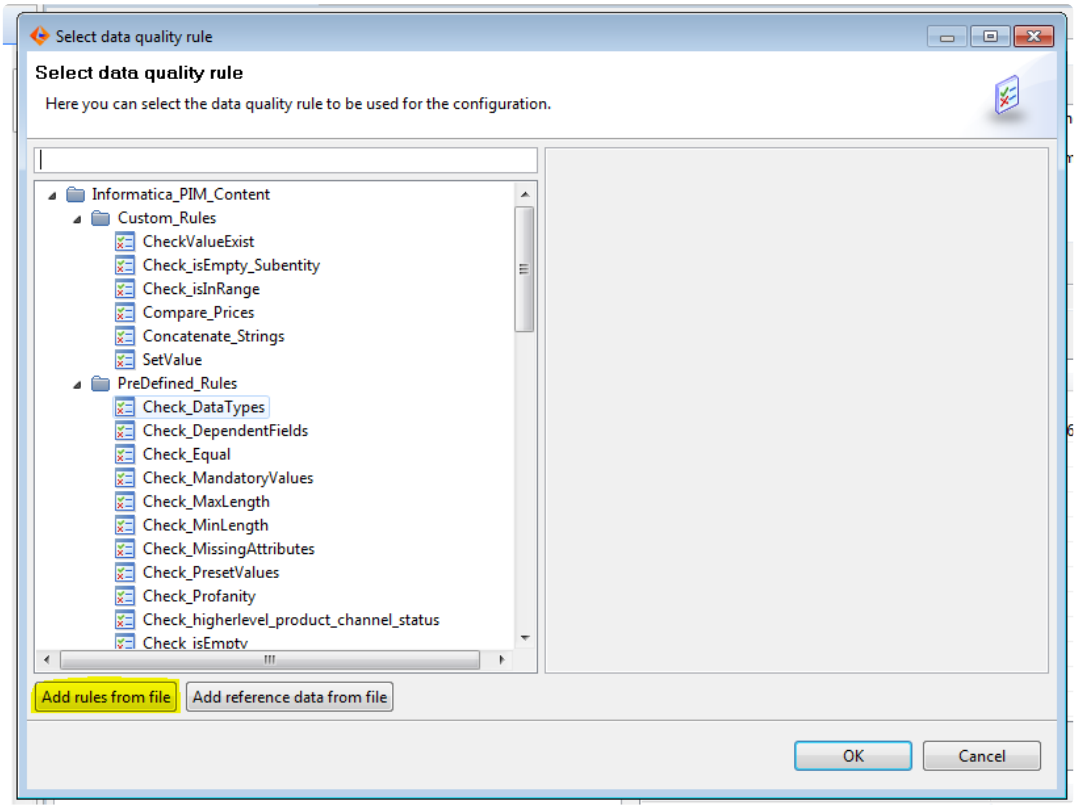
Make sure custom rules are exported with a **different XML filename than the standard rules (Informatica_PIM_Content.xml)**. You can have either one XML for all custom rules (e.g. “Custom_Rules_of_MyEnterprise.xml”) or even better one XML for each of them (“my_color_standardization.xml”, etc.). The export of custom rules should not include the **PreDefined_Rules** folder of the project so you can update the pre-built rules with any Product 360 upgrade and without any additional effort just by using the XML shipped with the install packages.

On the Product 360 Server the "dataquality" folder should be backed up now.

To upload the new XML version of your rules open the Desktop UI and navigate to the DQ perspective to open up the rule configuration dialog via the “…” button:




The dialog will allow you to upload the new XML files:




If you would have updated meta data as well you can additionally use the “Add reference data…” button from above screen to import them. If not you can skip that step (only needed if the reference data has changed).

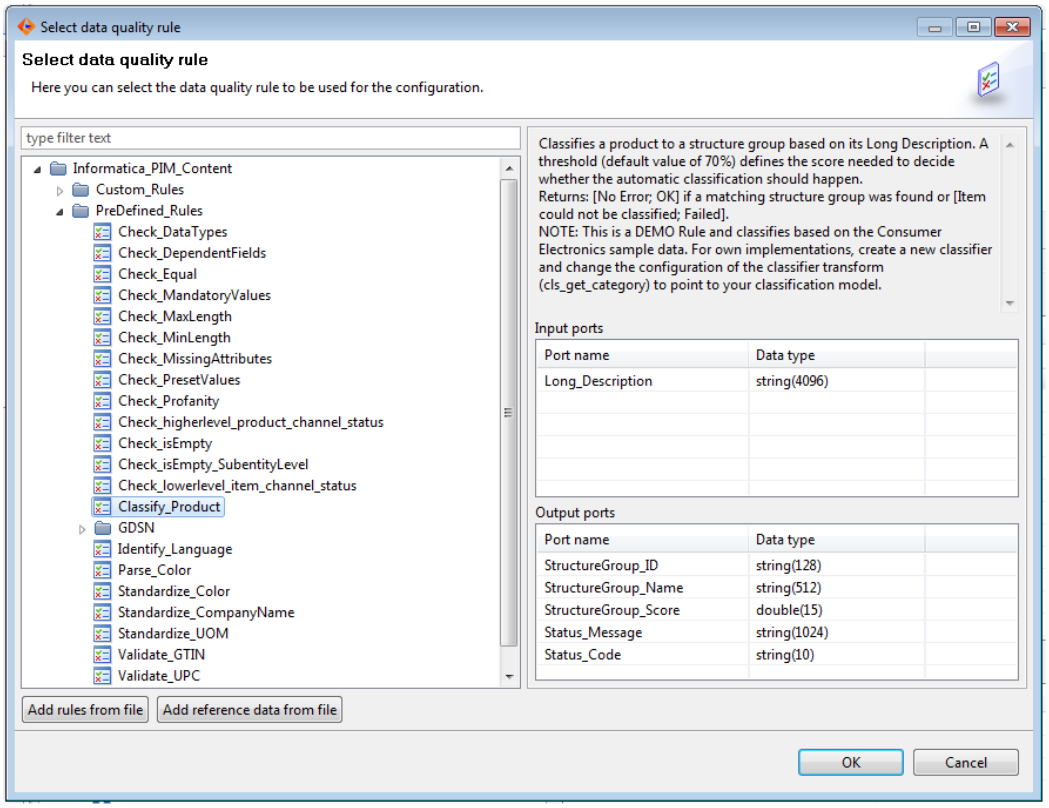
Upload all custom rule XML files from the upgraded IDQ environment.

Do this also with the pre-built rules XML (“Informatica_PIM_Content.xml”) shipped with the version of Product 360 that carries the IDQ SDK update so you have the latest and greatest of the out of the box rules as well.

Lastly go to your "dataquality" folder on the server and check for any potential duplicate or dated XML files to remove them and avoid duplicate rules (remember to backup first). The Date modified may serve as hint on whether this is a new XML or a dated one. In case you found duplicates or dated rules it is recommended to perform a server restart in the end so any memory load of rules on the server is also cleaned up.

# 2 Importing Rules into Product 360

In order to use the rules in Product 360 it is needed to upload them through the UI. In order to upload the files it is needed to create a Rule Configuration within the "Data quality configuration" view. When selecting a rule the following dialog pops up:




Add the XML file via the "Add rules from file" button and add the ZIP file via the "Add reference data from file" button.

# 3 Rule types and reference tables

## 3.1 Generic rules

Basic data quality rules that can be used in multiple use cases and scenarios to validate any data of products, variants and items in Product 360.

- Check_DependentFields
- Check_Equal
- Check_isEmpty
- Check_isEmpty_SubentityLevel (**since 8.1**)
- Check_MaxLength
- Check_MinLength

## 3.2 Structure Feature / Attribute rules

Data quality rules that can be used specifically for feature / attribute relational checks of products, variants and items stored in Product 360.

- Check_DataTypes
- Check_MandatoryValues
- Check_PresetValues
- Check_MissingAttributes

## 3.3 Product code validation rules

Data quality rules that can be used specifically to validate product code specific norms.

- Validate_GTIN
- Validate_UPC

## 3.4 Classification rules

Data quality rules that can be used specifically to classify objects based on the individual input values.

- Classify Product
- Identify_Language

## 3.5 Lookup and standardization rules

Data Quality rules that can be used specifically to standardize input values based on reference table lookups.

- Check_Profanity
- Parse_Color
- Standardize_Color
- Standardize_CompanyName
- Standardize_UOM

## 3.6 Reference tables

Reference tables are being used to store look up values and static information that will be accessed by the DQ rules during their runtime. Reference Tables that consist of one or two columns can be directly imported into Product 360 and maintained through the "Dictionary" perspective of the Desktop UI. Reference tables that consist of more than two columns will not show up on the Product 360 UI but directly be used by the DQ rules during runtime.

# 4 Generic Rules

Basic data quality rules that can be used in multiple use cases and scenarios to validate any product data.


- Check_DependentFields (see page 12)
- Check_Equal (see page 13)
- Check_isEmpty (see page 14)
- Check_isEmpty_SubentityLevel (see page 15)
- Check_MaxLength (see page 16)
- Check_MinLength (see page 18)


## 4.1 Check_DependentFields

### 4.1.1 General description

Checks if a dependent field has a value if its driver field is complete.

### 4.1.2 Input ports

| | |
|---|---|
| **inDriverField** | Data field that defines (based on whether it is empty or not) if the dependent field needs to be filled. |
| **inDependentField** | Dependent field which may not be empty if driver field has a value. |

### 4.1.3 Output ports

| | |
|---|---|
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.1.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.1.5 Example usage

If the attribute for "Hazardous Material" (inDriverField) is set to "true" the attribute value for "Risk & Security Sentences" (inDependentField) may not be empty

#### 4.1.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Checked field requires a value to be completed according to the dependent field. |

## 4.2 Check_Equal

### 4.2.1 General description

Checks if 2 field values assigned as input ports are equal.

### 4.2.2 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **inCheckField1** | Data field 1 that will be checked for equality against data field 2 |
| **inCheckField2** | Data field 2 that will be checked for equality against data field 1 |

### 4.2.3 Output ports

| | |
|---|---|
| **outObjectID** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple attribute checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |

| | |
|---|---|
| **Status_ Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.2.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.2.5 Example usage

Check if the value of Manufacturer (inCheckField1) is the same as for the attribute of brand name (inCheckField2)

#### 4.2.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | The input values are not equal. |

## 4.3 Check_isEmpty

### 4.3.1 General description

Checks if a given data field is null or empty.

### 4.3.2 Input ports

| | |
|---|---|
| **Field_Value** | Any product data field that will be checked whether it is 'null' or empty. |

### 4.3.3 Output ports

| | |
|---|---|
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |

| | |
|---|---|
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.3.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.3.5 Example usage

Check if the "English short description" (Field_Value) of an item is null or empty.

#### 4.3.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Input data value is blank or null. |

## 4.4 Check_isEmpty_SubentityLevel

### 4.4.1 General description

Checks if a given data field is null or empty. While running this rule on sub entities the inObjectID needs to be set to a valid object codeof the parent.

Note: sub entities need to exist to get evaluated by this rule, if no sub entities exists, this rule will never be executed and status will automatically be set to OK.

### 4.4.2 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **inCheckField** | Any data field on sub entity level that will be checked whether it is 'null' or empty. |

### 4.4.3 Output ports

| | |
|---|---|
| **Object_id** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple attribute checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.4.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.4.5 Example usage

Check if the any attribute value of an item is null or empty and report back failed if at least one of the attributes has no value set.

#### 4.4.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Input data value is blank or null. |

## 4.5 Check_MaxLength

### 4.5.1 General description

Checks if a given data field is exceeding the allowed length. The particular length can be set through the rule configuration user interface by typing the desired integer number into the "inLength" field.

### 4.5.2 Input ports

| | |
|---|---|
| **inLength** | User Input of maximum length allowed |
| **inData** | Data field that will be checked for its length |

### 4.5.3 Output ports

| | |
|---|---|
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.5.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.5.5 Example usage

Check if the "English short description" (inData) is longer than the defined maximum length (inLength) specified.

#### 4.5.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Data Field exceeds maximum length specified (10). |

## 4.6 Check_MinLength

### 4.6.1 General description

Checks if a given data field is having a minimum length. The particular length can be set through the rule configuration user interface by typing the desired integer number into the "inLength" field. Empty values are treated as zero-length and cause the rule to fail.

### 4.6.2 Input ports

| | |
|---|---|
| **inLength** | User Input of minimum length needed. |
| **inData** | Data field that will be checked for its length |

### 4.6.3 Output ports

| | |
|---|---|
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 4.6.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 4.6.5 Example usage

Check if the "English short description" (inData) is shorter than the allowed minimum length (inLength) specified.

#### 4.6.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Data field shorter than minimum length specified (7). |

# 5 Feature / Attribute Rules

Data quality rules that can be used specific for feature / attribute relational checks of products, variants and items stored in Product 360.


- Check_DataTypes (see page 20)
- Check_MandatoryValues (see page 22)
- Check_PresetValues (see page 25)
- Check_MissingAttributes (see page 28)


## 5.1 Check_DataTypes

### 5.1.1 General description

Check that the data type of an attribute is the same as the one of the assigned feature if the feature is not set to "character string" (e.g. The data type for the attribute "size" has to be set the same as for the assigned feature if the feature is not set to "Character String")

### 5.1.2 Sub-Entity type for Input

Attributes

### 5.1.3 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **inAttributeName** | Name of the attribute in the key language of the system. *(ArticleAttribute:name)* |
| **inLanguage** | Language of the specific attribute value. *(ArticleAttributeLang:language)* |
| **inStructure** | Structure system of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |
| **inStructureGroup** | Structure group of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |

| | |
|---|---|
| **inStructureFeature** | Feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap.StructureAttribute)* |
| **inAttributeValue** | Attribute value in its specific language. *(ArticleAttributeValue.Value)* |
| **inValueIdentifier** | Identifier of the attribute value. *(ArticleAttributeValue:identifier)* |
| **inStructurePresetValue** | Preset values defined by the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap.DomainValue)* |
| **inDataType** | Data type defined for the attribute. *(ArticleAttribute.Datatype)* |

### 5.1.4 Output ports

| | |
|---|---|
| **Object_id** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple attribute checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 5.1.5 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |
| **Language_Inclusion** | References all "languages" considered for the attribute value checks. The languages need to be written like their UI label text in the global standard language of the repository. (e.g. "English" and/or "German" if system language is set to "en" for English). |

| | |
|---|---|
| **Structure_checks_inclusion** | References the "identifier" values for each Structure System of Product 360 considered by the Rule. |
| **System_Language** | References the global standard language of the Product 360 repository (e.g. "en" for English) using a single line. Language needs to be value recognized by Product 360. |
| **java_structure_name_lookup** | Java transformation that synchronizes the database internal and external identifier (that is visible through the UI) of a structure system during rule execution. |
| **java_CheckAttribute** | Java Transformation that contains all the logic to gather the required values of the feature each attribute is assigned to. |

### 5.1.6 Example usage

Check for all attributes of an item (that are assigned to a feature of a structure system mentioned in "Structure_checks_inclusion") whether their data type is the same as the data type of the assigned feature if the feature isn't set to "Character String".

#### 5.1.6.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Pattern: *[Structure system], [Structure group]*, Feature '*[Feature name]*': The data type (*[Attribute data type]*) of attribute '*[Attribute name]*' does not match the assigned feature data type (*[Feature data type]*). *[line break; optional]*<br><br>Sample Data Consumer Electronics, HDTV Series, Feature 'USB 2.0':The data type (Character string) of attribute 'USB 2.0' does not match the assigned feature data type (Integer). |

## 5.2 Check_MandatoryValues

### 5.2.1 General description

Check that the attribute has a value if assigned to a feature that is defined as mandatory (e.g. a T-Shirt item needs to have a value for the attribute "Size" if this is a mandatory feature in the "T-Shirt" structure group)

### 5.2.2 Sub-Entity type for Input

Attributes

### 5.2.3 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **inAttributeName** | Name of the attribute in the key language of the system. *(ArticleAttribute:name)* |
| **inLanguage** | Language of the specific attribute value. *(ArticleAttributeLang:language)* |
| **inStructure** | Structure system of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |
| **inStructureGroup** | Structure group of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |
| **inStructureFeature** | Feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap.StructureAttribute)* |
| **inAttributeValue** | Attribute value in its specific language. *(ArticleAttributeValue.Value)* |
| **inValueIdentifier** | Identifier of the attribute value. *(ArticleAttributeValue:identifier)* |
| **inStructurePresetValue** | Preset values defined by the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap.DomainValue)* |

### 5.2.4 Output ports

| | |
|---|---|
| **Object_id** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple attribute checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 5.2.5 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |
| **Language_Inclusion** | References all "languages" considered for the attribute value checks. The languages need to be written like their UI label text in the global standard language of the repository. (e.g. "English" and/or "German" if system language is set to "en" for English). |
| **Structure_checks_inclusion** | References the "identifier" values for each Structure System of Product 360 considered by the Rule. |
| **System_Language** | References the global standard language of the Product 360 repository (e.g. "en" for English) using a single line. Language needs to be value recognized by Product 360. |
| **java_structure_name_lookup** | Java transformation that synchronizes the database internal and external identifier (that is visible through the UI) of a structure system during rule execution. |
| **java_CheckAttribute** | Java Transformation that contains all the logic to gather the required values of the feature each attribute is assigned to. |

### 5.2.6 Example usage

Check for all attributes of an item (that are assigned to a feature of a structure system mentioned in "Structure_checks_inclusion") whether they have a value (in the languages mentioned in "Language_Inclusion") if the assigned feature is defined as mandatory.

#### 5.2.6.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Pattern: *[Structure system]*, *[Structure group]*, Feature '*[Feature name]*': The mandatory value *[Language]* of attribute '*[Attribute name]*' is missing. *[line break; optional]*<br><br>Sample Data Consumer Electronics, MultyVision ISIO, Feature 'Color': The mandatory value (French) of attribute 'Color' is missing. |

## 5.3 Check_PresetValues

### 5.3.1 General description

Check that the value of an attribute is conform to the preset values defined for the assigned feature (e.g. The value for the attribute "Size" may only be "S", "M", "L" or "XL" according to the preset values given by the assigned feature)

### 5.3.2 Sub-Entity type for Input

Attributes

### 5.3.3 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **inAttributeName** | Name of the attribute in the key language of the system. *(ArticleAttribute:name)* |
| **inLanguage** | Language of the specific attribute value. *(ArticleAttributeLang:language)* |

| | |
|---|---|
| **inStructure** | Structure system of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |
| **inStructureGroup** | Structure group of the feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap:structure)* |
| **inStructureFeature** | Feature the attribute is assigned to. *(ArticleAttributeStructureGroupAttributeMap.StructureAttribute)* |
| **inAttributeValue** | Attribute value in its specific language. *(ArticleAttributeValue.Value)* |
| **inValueIdentifier** | Identifier of the attribute value. *(ArticleAttributeValue:identifier)* |
| **inStructurePresetValue** | Optional different value for the attribute. *(ArticleAttributeStructureGroupAttributeMap.DomainValue)* |

### 5.3.4 Output ports

| | |
|---|---|
| **Object_id** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple attribute checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 5.3.5 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

| | |
|---|---|
| **Language_Inclusion** | References all "languages" considered for the attribute value checks. The languages need to be written like their UI label text in the global standard language of the repository. (e.g. "English" and/or "German" if system language is set to "en" for English). |
| **Structure_checks_inclusion** | References the "identifier" values for each Structure System of Product 360 considered by the Rule. |
| **System_Language** | References the global standard language of the Product 360 repository (e.g. "en" for English) using a single line. Language needs to be value recognized by Product 360. |
| **java_structure_name_lookup** | Java transformation that synchronizes the database internal and external identifier (that is visible through the UI) of a structure system during rule execution. |
| **java_CheckAttribute** | Java Transformation that contains all the logic to gather the required values of the feature each attribute is assigned to. |

### 5.3.6 Example usage

Check for all attributes of an item (that are assigned to a feature of a structure system mentioned in "Structure_checks_inclusion") whether their preset value is similar to one of the values from the list of the assigned feature.

#### 5.3.6.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Pattern: *[Structure system], [Structure group]*, Feature '*[Feature name]*: The value '*[Attribute value]*' (*[Language]*, *[Identifier of the value]*) of attribute '*[Attribute name]*' doesn’t conform to the preset values. *[line break; optional]*<br><br>Sample Data Consumer Electronics, HDTV Series, Feature 'USB 2.0': The value 'enabled' (English, DEFAULT) of attribute 'USB 2.0' doesn’t conform to the preset values. |

## 5.4 Check_MissingAttributes

### 5.4.1 General description

Checks that an item has attributes for all features defined by the structure group it is assigned to (e.g. a T-Shirt needs to have an attribute for "Size" if this is a feature in the "T-Shirt" structure group)

### 5.4.2 Sub-Entity type for Input

Structure assignments

### 5.4.3 Input ports

| | |
|---|---|
| **inObjectID** | Identifier of the parent object. *(Object_ID)* |
| **Structure** | Identifier of the assigned Structure. *(ArticleStructureMap:structureId)* |
| **StructureGroup** | Structure group the object is assigned to. *(ArticleStructureMap.StructureGroup)* |

### 5.4.4 Output ports

| | |
|---|---|
| **Object_id** | Returns the identifier of the parent object. *(needed to aggregate status messages from multiple structure assignment checks to a single parent object status message)* |
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 5.4.5 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |
| **Structure_checks_inclusion** | References the "identifier" values for each Structure System of Product 360 considered by the Rule. |
| **System_Language** | References the global standard language of the Product 360 repository (e.g. "en" for English) using a single line. Language needs to be value recognized by Product 360. |
| **java_StructureIdentifierLookup** | Java transformation that synchronizes the database internal and external identifier (that is visible through the UI) of a structure system during rule execution. |
| **java_missing_attributes_check** | Java Transformation that contains all the logic to gather the required values of the assignment to identify missing attributes. |

### 5.4.6 Example usage

Check for all items (that have an assignment to a structure system mentioned in "Structure_checks_inclusion") whether they have a corresponding attribute for each of the features of their assigned structure group.

#### 5.4.6.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | Pattern: *[Structuresystem], [Structuregroup], Feature '[Feature name]*: There is no attribute assigned. *[line break; optional]*<br><br>Sample Data Home Improvement, Tool boxes, Feature 'Version': There is no attribute assigned.<br>Sample Data Home Improvement, Tool boxes, Feature 'Weight': There is no attribute assigned. |

# 6 Product Code Validation Rules

Data quality rules that can be used specifically to validate product code specific norms.


- Validate_GTIN (see page 30)
- Validate_UPC (see page 31)


## 6.1 Validate_GTIN

### 6.1.1 General description

Validates any given GTIN8, GTIN12, GTIN13 or GTIN14 number including length, check sum, etc.

### 6.1.2 Input ports

| | |
|---|---|
| **In_Field_Value** | Any data field with a GTIN. |

### 6.1.3 Output ports

| | |
|---|---|
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 6.1.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 6.1.5 Example usage

Check if the GTIN (In_Field_Value) of an item is valid and consistent.

#### 6.1.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | GTIN / EAN value is not valid. |

## 6.2 Validate_UPC

### 6.2.1 General description

Validates any given Universal Product Code (UPC) number.

### 6.2.2 Input ports

| | |
|---|---|
| **Field_Value** | Any data field with a UPC. |

### 6.2.3 Output ports

| | |
|---|---|
| **Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 6.2.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 6.2.5 Example usage

Check if the UPC (Field_Value) of an item is valid and consistent.

#### 6.2.5.1 Example output

| | |
|---|---|
| **Status_Code** | Failed |
| **Status_Message** | UPC value is not valid. |

# 7 Classification Rules

Data quality rules that can be used specifically to classify objects based on their individual input values.


- Classify_Product (see page 33)
- Identify_Language (see page 35)


## 7.1 Classify_Product

### 7.1.1 General description

Classifies a product to a structure group based on its "Long Description". A threshold (default value of 70%) defines the score needed to decide whether the classification should happen or not.

NOTE: This is a DEMO rule and classifies based on the "Sample Data Consumer Electronics" demo data. For own implementations, create a new classifier and change the configuration of the classifier transform (cls_get_category) to point to your classification model.

NLP Classifiers need to be trained with actual live data. This happens by mapping e.g. long descriptions of products to structure groups of Product 360. The broader the variety of long descriptions that have been mapped to a specific structure group value the better is the classification of new items working.

### 7.1.2 Input ports

| | |
|---|---|
| **Long_Description** | Long description that will be used to classify the object against a certain structure group. |

### 7.1.3 Output ports

| | |
|---|---|
| **StructureGroup_ID** | Returns the identifier of the structure group found for the product. |
| **StructureGroup_Name** | Returns the English name of the structure group found for the product. |
| **StructureGroup_Score** | Returns the probability score for the classification to the structure group found. *(QualityStatusEntry.Score)* |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |

| | |
|---|---|
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 7.1.4 Meta data and reference tables

| | |
|---|---|
| **cs_classify_products_demo** | Sample classifier based on the Consumer Electronics "Sample Data Consumer Electronics" demo data. |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 7.1.5 Example usage

Check an item for a proposed structure group of the "Sample Data Consumer Electronics" structure system.

The "English Long Description" (Long_Description) of the item is:

> *"46 inch multimedia television set (Full-HD) with high-quality aluminum front frame and contrast pane - comes equipped with Dual HDTV multi-tuner and digital video recorder (integrated hard drive)"*

Since the classifier of the rule has been trained with multiple possible long descriptions for all the structure groups of the system it suggests that the item should be assigned to the structure group called "HDTV Series"

#### 7.1.5.1 Example output

| | |
|---|---|
| **StructureGroup_ID** | 1267688379455 |
| **StructureGroup_Name** | HDTV Series |
| **StructureGroup_Score** | 1.0000 |
| **Status_Code** | OK |
| **Status_Message** | No Error |

## 7.2 Identify_Language

### 7.2.1 General description

Identifies the language of a text field by a probability score given after the identification. A threshold (default value of 70%) defines the score needed to decide whether the language classification was successful or not.

### 7.2.2 Input ports

| | |
|---|---|
| **Text_Value** | Text field that will be checked for its language. |

### 7.2.3 Output ports

| | |
|---|---|
| **Language** | Returns the language identified for the text field. |
| **Language_Score** | Returns the probability score for the identified language. *(QualityStatusEntry.Score)* |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 7.2.4 Meta data and reference tables

| | |
|---|---|
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 7.2.5 Example usage

Check whether the content of the "English Long Description" is actually written in English

The "English Long Description" (Text_Value) of the item is:

> *"46 inch multimedia television set (Full-HD) with high-quality aluminum front frame and contrast pane - comes equipped with Dual HDTV multi-tuner and digital video recorder (integrated hard drive)"*

#### 7.2.5.1 Example output

| | |
|---|---|
| **Language** | en |
| **Language_Score** | 0.9487 |
| **Out_Status_Code** | OK |
| **Out_Status_Message** | No Error |

# 8 Lookup and Standardization Rules

Data quality rules that can be used specifically to standardize input values based on reference table lookups.


- Check_Profanity (see page 37)
- Parse_Color (see page 38)
- Standardize_Color (see page 40)
- Standardize_CompanyName (see page 41)
- Standardize_UOM (see page 43)


## 8.1 Check_Profanity

### 8.1.1 General description

Parses inappropriate language and returns original text with profanity words removed.

### 8.1.2 Input ports

| | |
|---|---|
| **Text_Value** | Text field that will be checked for profanities. |

### 8.1.3 Output ports

| | |
|---|---|
| **Profanity_Value** | Returns the profanity value(s) found in the text. |
| **Masked_Data** | Returns the input text and exchanges profanity words with the value [CENSORED]. |
| **Cleansed_Text** | Returns the input text with the profanity words removed. |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 8.1.4 Meta data and reference tables

| | |
|---|---|
| **profanity_infa** | Dictionary that contains a variety of profanity values that are being used during rule execution for look ups. |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 8.1.5 Example usage

Check whether the "English Long Description" (Text_Value) contains any profanity values referenced by "profanity_infa".

The "English Long Description" of the item is:

*"HDTV with a crappy 50 GB hard drive"*

#### 8.1.5.1 Example output

| | |
|---|---|
| **Profanity_Value** | crappy |
| **Masked_Data** | HDTV with a CENSORED 50 GB hard drive |
| **Cleansed_Text** | HDTV with a 50 GB hard drive |
| **Status_Code** | Failed |
| **Status_Message** | Profanity -crappy- was found in input data. |

## 8.2 Parse_Color

### 8.2.1 General description

Parses color names as determined by a reference table. The rule will return 2 color values – for each, as found in the data as well as a cleansed / cased version.

### 8.2.2 Input ports

| | |
|---|---|
| **Text_Value** | Text field that will be parsed for color values. |

### 8.2.3 Output ports

| | |
|---|---|
| **Standard_Text** | Returns the text field with standardized color values. |
| **Color_ID1** | Returns the name of the first color value found. |
| **Color_ID2** | Returns the name of the second color value found. |
| **Color_Standardized1** | Returns the standardized color value found for the first color. |
| **Color_Standardized2** | Returns the standardized color value found for the second color. |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 8.2.4 Meta data and reference tables

| | |
|---|---|
| **colors_infa** | Dictionary that contains the color values in a standardized format. |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 8.2.5 Example usage

Parse out two color values from the "English long description" (Text_Value) and exchange them with the standardized format of the same color (Standard_Text).

The "English Long Description" of the item is:

*"This nice HDTV has a ferrari red frame and a cool black screen color."*

#### 8.2.5.1 Example output

| | |
|---|---|
| **Standard_Text** | This nice HDTV has a Ferrari Red frame and a Cool black screen color. |
| **Color_ID1** | ferrari red |
| **Color_ID2** | cool black |
| **Color_Standardized1** | Ferrari Red |
| **Color_Standardized2** | Cool black |
| **Status_Code** | OK |
| **Status_Message** | No Error |

## 8.3 Standardize_Color

### 8.3.1 General description

Returns a base color value (Base_Color) for an input color value (Text_Value).

### 8.3.2 Input ports

| | |
|---|---|
| **Text_Value** | Color value that will be standardized. |

### 8.3.3 Output ports

| | |
|---|---|
| **Base_Color** | Returns the standardized color value. |

| | |
|---|---|
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 8.3.4 Meta data and reference tables

| | |
|---|---|
| **colors_base_infa** | Reference table that maps different color values to a standardized color value (e.g. "Deep sky blue" to "Blue") |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 8.3.5 Example usage

Take the color value provided by the manufacturer of the item and derive a base color value out of it that can be used for web shop search classification of that particular item.

The attribute "Color" provide by the supplier has the value "*Midnight black*".

#### 8.3.5.1 Example output

| | |
|---|---|
| **Base_Color** | Black |
| **Status_Code** | OK |
| **Status_Message** | No Error |

## 8.4 Standardize_CompanyName

### 8.4.1 General description

Standardizes a company name and additionally provides its acronym if possible.

### 8.4.2 Input ports

| | |
|---|---|
| **CompanyName** | Company name that will be standardized. |

### 8.4.3 Output ports

| | |
|---|---|
| **Standardized_CompanyName** | Returns the standardized company name. |
| **Acronym_ComapnyName** | Returns the acronym for the standardized company name. |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 8.4.4 Meta data and reference tables

| | |
|---|---|
| **usa_company_acronyms_infa** | Reference table that maps company names to their corresponding acronyms (e.g. "Hewlett-Packard Co" to "HP") |
| **usa_company_names_std_infa** | Reference table that maps company names to their standardized spelling format (e.g. "Abercrombie and Fitch" to "Abercrombie & Fitch Co") |
| **usa_company_sufx_abrv_infa** | Reference table that contains set of American suffixes for company names and their abbreviations (e.g. "Co.") |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 8.4.5 Example usage

Take the manufacturer name "Hewlett Packard" (CompanyName) attached to an item and standardize it to the defined format as determined by the reference table.

#### 8.4.5.1 Example output

| | |
|---|---|
| **Standardized_CompanyName** | Hewlett-Packard |
| **Acronym_CompanyName** | HP |
| **Status_Code** | OK |
| **Status_Message** | No Error |

## 8.5 Standardize_UOM

### 8.5.1 General description

Separates the quantity and unit of measure, and outputs the unstandardized and standardized values. It also outputs the full string with the Unit of Measure standardized.

### 8.5.2 Input ports

| | |
|---|---|
| **Text_Value** | Text field to be checked for unit of measures. |

### 8.5.3 Output ports

| | |
|---|---|
| **Standardized_Unit** | Returns the standardized unit of measure found in the text. |
| **Parsed_Unit** | Returns the unit of measure found in the text. |
| **Standardized_Text** | Returns the text field with standardized UOM values. |
| **Additional_Parsed_Values** | Returns any additional parsed values found in the text. |

| | |
|---|---|
| **Unparsed_Field** | Returns the part of the text that hasn't been parsed by the rule. |
| **Out_Status_Code** | Returns the overall Status Code after the rule execution (OK or Failed). *(QualityStatusEntry.Status)* |
| **Out_Status_Message** | Returns the overall Status Message after the rule execution. *(QualityStatusEntry.Message)* |

### 8.5.4 Meta data and reference tables

| | |
|---|---|
| **uom_infa** | Reference table that contains the unit values in a standardized format. |
| **Error_messages_by_Language** | Contains a 4 digit error code plus a language to indicate the preferred error message to be output. |

### 8.5.5 Example usage

The "English Long Description" of the item that will be parsed by that rule is:

*"This nice HDTV weighs 10 kilogram and can be delivered in 24 hours."*

#### 8.5.5.1 Example output

| | |
|---|---|
| **Standardized_Unit** | 10 kg |
| **Parsed_Unit** | 10 kilogram |
| **Standardized_Text** | This nice HDTV weighs 10 kg and can be delivered in 24 hrs. |
| **Additional_Parsed_Values** | 24 hours |
| **Unparsed_Field** | This nice HDTV weighs and can be delivered hours. |

| | |
|---|---|
| **Status_Code** | OK |
| **Status_Message** | No Error |

# 9 Reference Tables

Reference tables are being used to store look up values and static information that will be accessed by the DQ rules during their runtime. Reference Tables that consist of one or two columns can be directly imported into Product 360 and maintained through the "Dictionary" perspective of the rich client. Reference tables that consist of more than two columns will not show up on the Product 360 UI but directly be used by the DQ rules during runtime.


- Product 360 administrative reference tables (see page 46)
- Reference tables for data look ups and standardizations (see page 49)


## 9.1 Product 360 administrative reference tables

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>Custom_Error_messages_by_language</td><td>Reference table that contains the error messages for custom rules.<br><br>Important: Please add error messages for custom rules always into this table.<br><br>Contains a 4 digit error code plus a language to indicate the preferred error message to be output.<br><br><table><tr><th>Field</th><th>Description</th></tr><tr><td>Error_message</td><td>Text that builds up the error message of a custom DQ rule in case of a "failed" status result.</td></tr><tr><td>Error_code</td><td>Identifier that is used by the custom DQ rule to reference the error_message.</td></tr></table></td><td>Yes</td></tr>
</table>

<table>
<tr>
<td>Error_messages_by_Language</td>
<td>Reference table that contains the error messages for standard rules.<br><br>Contains a 4 digit error code plus a language to indicate the preferred error message to be output.<br><br>
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>Error_message</td><td>Text that builds up the error message of a DQ rule in case of a "failed" status result.</td></tr>
<tr><td>Error_code</td><td>Identifier that is used by the DQ rule to reference the error_message.</td></tr>
</table>
</td>
<td>Yes</td>
</tr>
<tr>
<td>Language_Inclusion</td>
<td>References all "languages" considered for the attribute value checks. The languages need to be written like their UI label text in the global standard language of the repository. (e.g. "English" and "German" if system language is set to "en, de").<br><br>
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>language_list</td><td>Semi colon separated list of languages that are included into the DQ checks that use this reference table (e.g. "English;German;French").</td></tr>
<tr><td>lookup_val</td><td>Fixed string "lang_list" that is used as an identifier for the DQ rules.</td></tr>
</table>
</td>
<td>Yes</td>
</tr>
</table>

<table>
<tr>
<td>Structure_checks_inclusion</td>
<td>References the "identifier" values for each Structure System of Product 360 considered by rules that do structure/feature/attribute checks.
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>flag</td><td>Indicates whether the structure system will be taken into consideration by the DQ rule that uses this reference table.</td></tr>
<tr><td>Structure_identifier</td><td>List of all structure system identifiers that could be taken into consideration by the DQ rule that uses this reference table.</td></tr>
</table>
</td>
<td>Yes</td>
</tr>
<tr>
<td>System_Language</td>
<td>References the global standard language of the Product 360 repository (e.g. "en" for English) using a single line. Language needs to be value recognized by Product 360.
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>SysLang</td><td>The short spelling of the overall system language / repository language (e.g. "en" for English). This will have effect on data access through Web API during rule execution and the language the error messages are written back into the object status.</td></tr>
<tr><td>Lookup_value</td><td>Fixed string "sys_lang" that is used as an identifier for the DQ rules.</td></tr>
</table>
</td>
<td>Yes</td>
</tr>
</table>

## 9.2 Reference tables for data look ups and standardizations

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>color_base_infa</td><td>Reference table that maps different color values to a standardized color value (e.g. "Deep sky blue" to "Blue")<table><tr><th>Field</th><th>Description</th></tr><tr><td>column1</td><td>The base color value (e.g. "Blue") a DQ rule that uses this table will take for standardization.</td></tr><tr><td>column2</td><td>The variation of the color value (e.g. "Deep sky blue") a DQ rule uses as look up for a base color value return.</td></tr></table></td><td>Yes</td></tr>
</table>

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>colors_infa</td><td>Dictionary that contains the color values in a standardized format.<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>column1</td><td>The base color value (e.g. "Blue") a DQ rule that uses this table will look up to identify a color string.</td></tr>
<tr><td>column2</td><td>Variation of the base color.</td></tr>
<tr><td>column3</td><td>Variation of the base color.</td></tr>
<tr><td>column4</td><td>Variation of the base color.</td></tr>
<tr><td>column5</td><td>Variation of the base color.</td></tr>
<tr><td>column6</td><td>Variation of the base color.</td></tr>
</table></td><td>No</td></tr>
</table>

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>profanity_infa</td><td>Dictionary that contains a variety of profanity values that are being used during rule execution for look ups.<table><tr><th>Field</th><th>Description</th></tr><tr><td>COLUMN1</td><td>The profanity word a DQ rule that uses this table will look up to identify bad words.</td></tr><tr><td>COLUMN2</td><td>The profanity word a DQ rule that uses this table will look up to identify bad words (the same value as in column 1).</td></tr></table></td><td>Yes</td></tr>
<tr><td>uom_infa</td><td>Reference table that contains the unit values in a standardized format.<table><tr><th>Field</th><th>Description</th></tr><tr><td>column1</td><td>The base unit (e.g. "sec") a DQ rule that uses this table will look up to identify unit.</td></tr><tr><td>column2</td><td>Variation of the base unit.</td></tr><tr><td>column3</td><td>Variation of the base unit.</td></tr><tr><td>column4</td><td>Variation of the base unit.</td></tr><tr><td>column5</td><td>Variation of the base unit.</td></tr></table></td><td>No</td></tr>
</table>

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>usa_company_acronyms_infa</td><td>Reference table that maps company names to their corresponding acronyms (e.g. "Hewlett-Packard Co" to "HP")<table><tr><th>Field</th><th>Description</th></tr><tr><td>column1</td><td>The abbreviation of a given company name (e.g. "hp") a DQ rule uses as look up for a standardized company name value return.</td></tr><tr><td>column2</td><td>The  standardized company name (e.g. "Hewlett Packard") a DQ rule that uses this table will take for standardization.</td></tr></table></td><td>Yes</td></tr>
</table>

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>usa_company_names_std_infa</td><td>Reference table that maps company names to their standardized spelling format (e.g. "Abercrombie and Fitch" to "Abercrombie & Fitch Co")<table><tr><th>Field</th><th>Description</th></tr><tr><td>column1</td><td>The standardized company name (e.g. "Hewlett Packard") a DQ rule that uses this table will take for standardization.</td></tr><tr><td>column2</td><td>Variation of the company name.</td></tr><tr><td>column3</td><td>Variation of the company name.</td></tr><tr><td>column4</td><td>Variation of the company name.</td></tr><tr><td>column5</td><td>Variation of the company name.</td></tr><tr><td>column6</td><td>Variation of the company name.</td></tr><tr><td>etc.</td><td>etc.</td></tr></table></td><td>No</td></tr>
</table>

<table>
<tr><th>Name</th><th>Description</th><th>Importable into Product 360?</th></tr>
<tr><td>usa_company_sufx_abrv_infa</td><td>Reference table that contains set of American suffixes for company names and their abbreviations (e.g. "Co.")
<table>
<tr><th>Field</th><th>Description</th></tr>
<tr><td>column1</td><td>The standardized company suffix (e.g. "Inc.") a DQ rule that uses this table will take for standardization.</td></tr>
<tr><td>column2</td><td>Variation of the company suffix .</td></tr>
<tr><td>column3</td><td>Variation of the company suffix .</td></tr>
<tr><td>column4</td><td>Variation of the company suffix .</td></tr>
<tr><td>column5</td><td>Variation of the company suffix .</td></tr>
<tr><td>column6</td><td>Variation of the company suffix .</td></tr>
<tr><td>etc.</td><td>etc.</td></tr>
</table></td><td>No</td></tr>
</table>

# Copyright

© Copyright Informatica LLC 1993, 2025

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.